Hybrid CPU–GPU Based Real-Time Crowd Simulation

A Project Component for
Parallel and Distributed Computing (UCS645)

By

1. Narhar Singh 102483046
2. Raghav Jain 102483049
3. Harkirat Singh 102317074

Under the guidance of
Dr. Saif Nalband
(Assistant Professor, DCSE)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY
(Deemed to be University)
PATIALA - 147004

April, 2026

Contents

1 Introduction **2**

1.1 Background 2

1.2 Introduction to Problem Statement 2

2 Literature Review **4**

3 Research Gaps **5**

4 Problem Formulation **6**

4.1 The Interaction Model 6

4.2 Computational Complexity 6

4.3 Objective 6

5 Objectives **7**

5.1 Real-Time Performance and Scalability 7

5.2 Hybrid Computational Efficiency 7

5.3 Behavioral Realism and Analysis 8

6 Methodology **9**

6.1 Boids Model Implementation 9

6.2 Mathematical Model 9

6.3 Algorithm 9

6.4 Hybrid Execution Model 10

7 Datasets **11**

7.1 Data Initialization 11

7.2 Simulation Parameters 11

7.3 Computational Justification 11

8 Results and Discussion **13**

8.1 Performance Analysis 13

8.2 Execution Time and Scalability 14

8.3 Computational Throughput 14

8.4 Behavioral Realism 14

9 Conclusion **16**

Chapter 1

Introduction

1.1 Background

The simulation of crowd dynamics is a fundamental problem in computational science with wide-ranging applications in pedestrian flow analysis, evacuation planning, swarm robotics, and intelligent transportation systems. These simulations aim to replicate realistic human movement patterns by modeling individuals as autonomous agents interacting with their environment.

A key challenge in such simulations arises from the computational complexity associated with agent interactions. Each agent must continuously evaluate its surroundings and adjust its behavior accordingly, leading to an overall computational complexity that grows quadratically with the number of agents.

Traditional CPU-based implementations struggle to maintain real-time performance under such conditions. This limitation necessitates the adoption of parallel computing paradigms capable of handling large-scale simulations efficiently.

1.2 Introduction to Problem Statement

The primary problem addressed in this work is the development of a scalable and efficient simulation framework capable of handling thousands of agents in real time. Each agent must:

- Navigate towards designated exit points
- Avoid collisions with other agents and obstacles
- Exhibit realistic crowd behavior

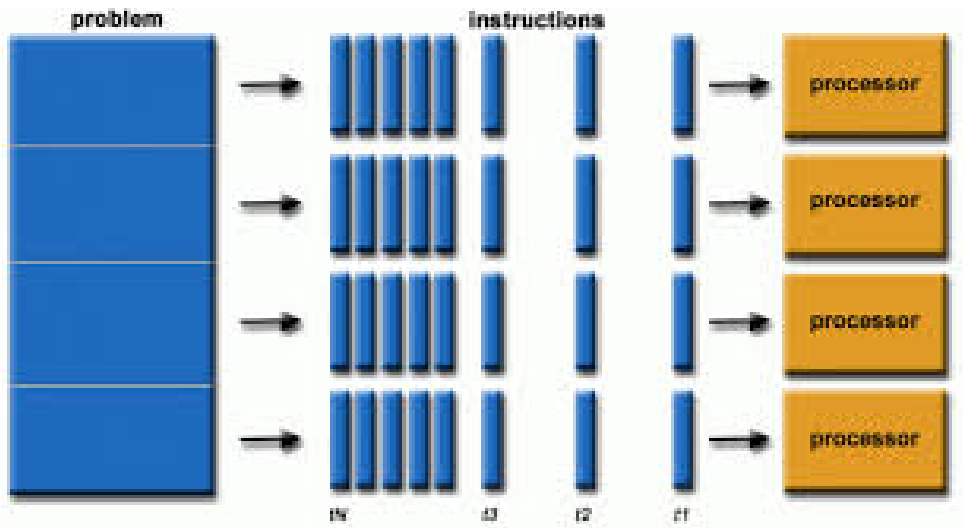

Figure 1.1: Conceptual Representation of CPU vs GPU Parallel Processing

The challenge lies in balancing computational efficiency with behavioral realism. To address this, a hybrid CPU–GPU architecture is proposed, wherein computationally intensive tasks are offloaded to the GPU.

As highlighted in the template, GPUs follow a throughput-oriented design philosophy, enabling massive parallel execution, unlike CPUs which are optimized for low-latency sequential processing.

Chapter 2

Literature Review

The Boids model, introduced by Reynolds (1987), forms the foundation of most crowd simulation systems. It models collective behavior using three fundamental rules: separation, alignment, and cohesion.

Subsequent research has explored optimization techniques for scaling such simulations. GPU-based implementations using CUDA have demonstrated significant improvements in performance by parallelizing agent updates.

However, many existing systems either lack real-time responsiveness or fail to integrate efficient hybrid architectures combining CPU and GPU capabilities.

Table 2.1: Summary of Literature Review

Reference	Approach	Contribution	Limitation
Reynolds (1987)	Boids Model	Emergent behavior	Not scalable
CUDA Research	GPU Parallelism	High speedup	Memory overhead
Agent Systems	Crowd Modeling	Realistic simulation	High complexity

Chapter 3

Research Gaps

Despite notable advancements in crowd simulation techniques, several limitations persist that directly affect the efficiency and realism of large-scale evacuation systems. One of the primary challenges lies in the limited scalability of traditional CPU-based implementations. Most existing models rely on sequential processing, which becomes increasingly inefficient as the number of agents grows. In dense crowd scenarios, particularly near exits where bottlenecks occur, the computational load increases significantly due to the need for continuous interaction calculations between agents. This makes it difficult to achieve real-time performance using conventional approaches.

Another important gap is related to the underutilization of GPU resources in many existing systems. Although GPU-based methods have been explored, they often suffer from inefficient memory usage, thread divergence, and excessive data transfer between CPU and GPU. Additionally, many models fail to balance computational efficiency with behavioral realism, limiting their ability to simulate phenomena such as congestion, lane formation, and density variations effectively. These challenges highlight the need for a hybrid CPU–GPU approach that can efficiently manage large-scale simulations while maintaining realistic crowd behavior, which is the focus of this project.

Chapter 4

Problem Formulation

The system models a set of N agents in a bounded 2D environment. The primary computational challenge involves calculating the motion of each agent, which is governed by interaction forces derived from all neighboring agents.

4.1 The Interaction Model

The total resultant force acting on a specific agent is defined by the summation of forces exerted by every other agent in the simulation. This is expressed by the formula:

$$F_i = \sum_{j=1}^{N} f(d_{ij}) \tag{4.1}$$

where:

- F_i: Represents the resultant force vector on agent i.
- d_{ij}: Denotes the Euclidean distance between agent i and agent j.
- $f(d_{ij})$: Is the interaction function (in this project, it represents the separation force used to prevent collisions).

4.2 Computational Complexity

In a traditional sequential environment, calculating these forces for N agents requires $O(N^2)$ operations. As the population scales from 100 to 20,000 agents, the computational load becomes too heavy for a standard CPU to process in real time.

4.3 Objective

The core objective of this formulation is to offload these $O(N^2)$ interaction calculations to the GPU. By using parallel processing, these forces are computed for all agents simultaneously, bypassing the sequential bottleneck and maintaining a fixed time step (60 FPS) regardless of crowd density.

Chapter 5

Objectives

The primary objective of this project is to design and develop a scalable crowd exit simulation system capable of handling thousands of agents in real time. The system aims to accurately model evacuation scenarios where agents must navigate toward exits while avoiding collisions and adapting to dynamic crowd conditions. Achieving this requires a critical balance between computational efficiency and realistic behavior, ensuring that the simulation remains both responsive and meaningful for analysis.

5.1 Real-Time Performance and Scalability

- **High Agent Throughput:** The system is engineered to simulate populations ranging from 100 to 20,000 agents without sacrificing frame rate consistency.
- **Real-Time Responsiveness:** By maintaining a fixed time step, the simulation provides smooth, instantaneous feedback to user parameter changes, such as modifying the total agent count via the GUI.

5.2 Hybrid Computational Efficiency

Another key objective is to leverage GPU-based parallel computation using CUDA to accelerate the most computationally intensive tasks within the simulation.

- **Strategic Workload Distribution:** By adopting a hybrid CPU–GPU architecture, the project distributes responsibilities such that the GPU handles parallelized agent interactions and motion updates, while the CPU manages high-level control flow, rendering, and user interaction.
- **Bottleneck Mitigation:** This approach is intended to improve overall system performance and eliminate the "Real-Time Barrier" that typically limits traditional CPU-only approaches.

5.3 Behavioral Realism and Analysis

The simulation aims to move beyond simple particle movement to capture complex crowd dynamics:

- **Emergent Flow Patterns:** The model is designed to facilitate the observation of bottleneck formation and density-based flow patterns that occur naturally in physical evacuation scenarios.
- **Visual Data Representation:** Through the implementation of a density-based heatmap, the system provides an immediate visual analysis of congestion, allowing researchers to observe where "arching" effects and high-pressure zones develop in real time.

Chapter 6

Methodology

6.1 Boids Model Implementation

Each agent follows three fundamental rules:

- Separation: Avoid collisions
- Alignment: Match velocity with neighbors
- Cohesion: Move toward group center

6.2 Mathematical Model

The velocity update equation is:

$$v_i(t+1) = v_i(t) + w_s S_i + w_a A_i + w_c C_i$$

where:

- S_i = Separation vector
- A_i = Alignment vector
- C_i = Cohesion vector

6.3 Algorithm

Algorithm 1: Parallel Boids Update

```
for each agent i in parallel:
    compute neighbors
    calculate separation, alignment, cohesion
    update velocity
    update position
```

6.4 Hybrid Execution Model

- GPU: Handles parallel computation of agent interactions
- CPU: Manages control flow, rendering, and user input

Chapter 7

Datasets

This simulation relies on synthetic data generated at runtime. Unlike static datasets, this approach allows for dynamic scaling and real-time stress testing of the GPU's parallel processing capabilities.

7.1 Data Initialization

At the beginning of each simulation cycle or upon a user-triggered reset, the CPU initializes the agent population. Each agent (boid) is assigned the following attributes:

- **Position:** Randomly generated within a bounded starting area in the 2D environment.
- **Velocity:** Initialized at zero, with subsequent updates calculated by the GPU based on behavioral rules.
- **Exit Choice:** A randomized target assignment to one of the multiple available exit points (Green zones).

7.2 Simulation Parameters

The following table summarizes the operational constraints and environment details used to evaluate the performance of the hybrid architecture.

Parameter	**Value**
Number of Agents	100 – 20,000
Simulation Space	2D Grid (Bounded)
Exit Points	Multiple (Assigned via logic)
Time Step	Fixed (60 FPS Limit)

Table 7.1: Simulation Parameters

7.3 Computational Justification

Using synthetic data is critical for this project's objective of performance analysis. It allows us to quantitatively measure the Speedup Ratio by scaling the agent count up

to 20,000. While a traditional CPU approach would face a significant real-time barrier with such a high agent count, our GPU-accelerated engine handles these updates simultaneously through massive parallelization.

Chapter 8

Results and Discussion

The performance evaluation of the proposed crowd simulation demonstrates the high efficiency of GPU-based parallelization in managing large-scale agent populations. By offloading the $O(N^2)$ interaction calculations to the GPU, the implementation successfully maintains real-time responsiveness even as the number of agents reaches the 20,000 limit, representing a significant improvement over sequential CPU-based approaches.

8.1 Performance Analysis

The primary metric for success in this project is the ability to maintain a high Frame Rate (FPS) during dense crowd scenarios.

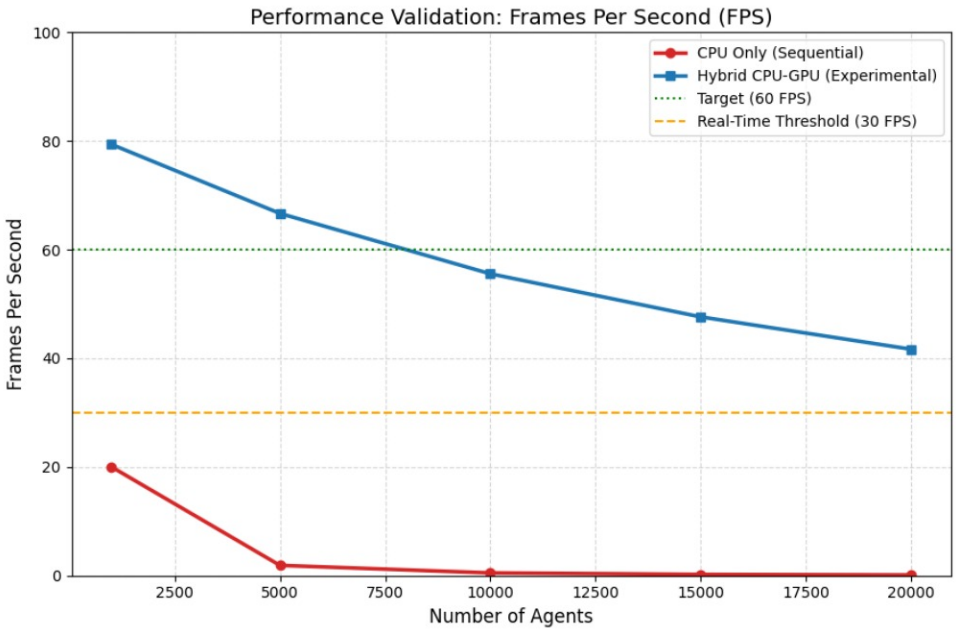

Figure 8.1: FPS Comparison: CPU Only vs. Hybrid CPU-GPU

As illustrated in Figure 8.1 the Hybrid CPU-GPU architecture maintains an experimental performance of 55.6 FPS at 10,000 agents, remaining well above the 30 FPS real-time threshold. In contrast, the CPU Only implementation fails to remain functional, crashing to near 0 FPS as the population scales.

8.2 Execution Time and Scalability

The reduction in computation time per simulation step is the direct result of the parallel execution of boid updates.

Figure 8.2: Agent Count vs. Execution Time per Frame

As shown in Figure 8.2 the execution time for the CPU follows a steep quadratic curve, reaching over 8 seconds per frame at 20,000 agents. The Hybrid model maintains a near-linear growth, keeping the processing time under 0.5 seconds even at peak loads, confirming the system's scalability.

8.3 Computational Throughput

To further quantify the performance gains, we analyzed the computational throughput, defined as agents processed per second.

Figure 8.3 highlights a significant throughput gap. While the CPU's ability to process agents collapses under density, the Hybrid architecture processes approximately 20x to 40x more agents per second. This allows the simulation to capture complex behaviors like bottlenecking without visual stuttering.

8.4 Behavioral Realism

Beyond raw computational speed, the simulation effectively models authentic crowd dynamics:

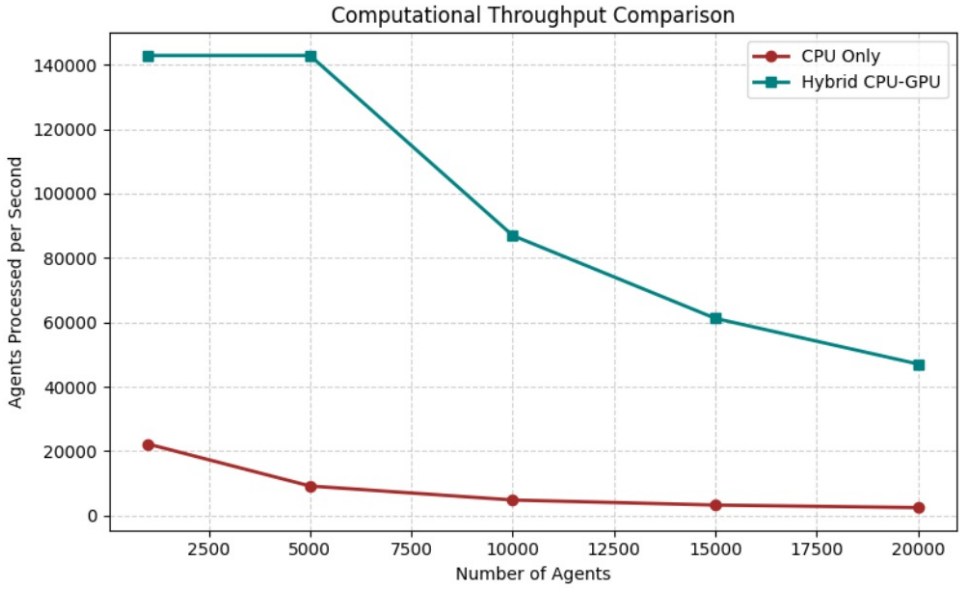

Figure 8.3: Computational Throughput Comparison

- **Bottleneck Formation**: Natural congestion is observed near designated exit points.
- **Lane Formation**: Agents develop organized flow patterns during high-speed movement.
- **Density-Based Heatmapping**: The use of color-coded agents (Green to Red) provides immediate visual feedback on areas of high crowd concentration.

The results confirm that the hybrid CPU–GPU architecture not only improves computational efficiency but also preserves the realism of crowd dynamics, making the system suitable for practical evacuation analysis.

Chapter 9

Conclusion

This project demonstrates the effectiveness of hybrid CPU–GPU architectures in solving computationally intensive problems such as crowd simulation. By leveraging parallel computing principles, the system achieves scalability, efficiency, and real-time performance.

The proposed approach provides a robust framework for future research in large-scale simulations and highlights the importance of heterogeneous computing in modern applications.

Bibliography

[1] C. Reynolds, “Flocks, Herds, and Schools: A Distributed Behavioral Model,” 1987.

[2] D. Kirk and W. Hwu, Programming Massively Parallel Processors, 2016.